Caso: Avaliação *holdout* de Classificador Binário na sklearn

Aula 06: Introdução ao Processamento de Linguagem Natural (PLN)

Prof. Eduardo Corrêa Gonçalves

26/03/2026

Sumário

Introdução

Avaliação *holdout* de classificadores binários: passo a passo

- Base de dados
- Divisão da base rotulada em treino e teste
- pré-processamento dos textos
- Treinamento do modelo com a partição de treino
- Teste do modelo com a partição de teste

Caso: Análise de Sentimentos

- **Apresentação do Exemplo**
  - Nesta aula, apresentaremos um exemplo de avaliação holdout de modelo de classificação binário na sklearn. O exemplo foi projetado da maneira a seguir:
    - **Base de dados (BD)**: "sentiment140", uma BD real para classificação de sentimentos
    - **Algoritmos que serão comparados**: Naïve Bayes Multinomial *versus* Regressão Logística (*a regressão logística é um exercício proposto !!!*)
    - **Estruturação do texto**: Bag of Words

Link para notebook com os exemplos da aula:

- O código apresentado nos slides a seguir está disponível no seguinte notebook:

- https://colab.research.google.com/drive/1_80Mx_d3qPaBcqOtxIuDyCoQJXBXuUJM?usp=sharing

Base de Dados

- **<u>Base:</u> Sentiment140**
  - Usaremos novamente a base para classificação de sentimentos **Sentiment140**
  - Possui 2 colunas:
    - 1ª - sentimento (*sentiment*)
    - 2ª - texto (review)
  - São 359 textos, sendo 182 positivos e 187 negativos
    - Base bem balanceada

	sentiment	review
0	positive	@stellargirl I loooooooovvvvvveee my Kindle2. ...
1	positive	Reading my kindle2... Love it... Lee childs i...
2	positive	Ok, first assesment of the #kindle2 ...it fuck...
3	positive	@kenburbary You'll love your Kindle2. I've had...
4	positive	@mikefish Fair enough. But i have the Kindle2...
...	...	...
354	positive	After using LaTeX a lot, any other typeset mat...
355	negative	On that note, I hate Word. I hate Pages. I hat...
356	positive	Ahhh... back in a *real* text editing environm...
357	negative	Trouble in Iran, I see. Hmm. Iran. Iran so far...
358	negative	Reading the tweets coming out of Iran... The w...

Avaliação Holdout na sklearn (1/14)

- **Os 6 passos para avaliar um classificador por holdout na sklearn:**
  1. Importação da base de dados rotulada.
  2. Divisão da base rotulada em treino e teste (partições: X_treino, Y_treino, X_teste, Y_teste)
  3. Pré-processamento das partes X de treino e X de teste.
  4. Treinamento do modelo com a partição de treino.
  5. Teste do modelo na partição de teste.

- Não é um esquema fixo! Dependendo do problema, você pode alterar esse fluxo.

  - **Exemplo:** Algumas vezes a base rotulada já é fornecida com as partições de treino e teste separadas. Neste caso, você não precisará do passo 3.

Avaliação Holdout na sklearn (2/14)

- **<u>PASSO 1</u>: importação da base de dados (BD) rotulada**
  - O arquivo CSV será importado para um **DataFrame** pandas

```
import pandas as pd

# ------------------------------------------------------------
# Importa os dados de treinamento para um DataFrame
# ------------------------------------------------------------
arq = https://raw.githubusercontent.com/edubd/bd/refs/heads/main/sentiment140.csv'

df = pd.read_csv(arq)

df # p/ visualizar o conteúdo
```

	sentiment	review
0	positive	@stellargirl I loooooooovvvvvveee my Kindle2. ...
1	positive	Reading my kindle2... Love it... Lee childs i...
2	positive	Ok, first assesment of the #kindle2 ...it fuck...
3	positive	@kenburbary You'll love your Kindle2. I've had...
4	positive	@mikefish Fair enough. But i have the Kindle2...
...	...	...
354	positive	After using LaTeX a lot, any other typeset mat...
355	negative	On that note, I hate Word. I hate Pages. I hat...
356	positive	Ahhh... back in a *real* text editing environm...
357	negative	Trouble in Iran, I see. Hmm. Iran. Iran so far...
358	negative	Reading the tweets coming out of Iran... The w...

359 rows × 2 columns

Avaliação Holdout na sklearn (3/14)

- **<u>PASSO 2 (parte 1)</u>: divisão da base de dados em treino e teste**
  - Primeiro separamos X_raw (textos) e Y (classes)

```
# Separação em: X_raw (X "cru", ainda não pré-processado)
#               Y     (classe)
X_raw = df.loc[:, ['review']] # DataFrame com a coluna "review"
Y = df.iloc[:, 0]  # Series com a coluna "sentiment"

print ("X_raw: ", X_raw.shape) # imprime num. de linhas e cols
print ("Y: ", Y.shape) # imprime tamanho do vetor
```

```
X_raw:  (359, 1)
Y:  (359,)
```

Avaliação Holdout na sklearn (4/14)

- **PASSO 2 (parte 2): divisão da base de dados em treino e teste**
  - Utiliza-se o método **train_test_split** (classe model_selection do módulo sklearn.model_selection)
  - Ela **vai gerar 4 estruturas**, já estratificadas por padrão (respeitando a proporção das classes):
    - X_raw_treino e Y_treino (que usaremos para treinar o classificador)
    - X_raw_teste e Y_teste (que usaremos para testar o classificador)
  - Os parâmetros train_size e test_size são usados para definir o tamanho das partições de treino e teste, respectivamente (nesse caso 67% e 33%).
  - O parâmetro random_state serve para definir uma semente que garantirá com que as mesmas partições sejam geradas sempre que executarmos o código

Avaliação Holdout na sklearn (5/14)

- **<u>PASSO 2 (parte 2)</u>: divisão da base de dados em treino e teste**
  - Abaixo o código para fazer a divisão

```
# separa X e Y em treino (67%) e teste (33%)
# de forma estratificada
from sklearn.model_selection import train_test_split

X_raw_treino, X_raw_teste, Y_treino, Y_teste =
model_selection.train_test_split(X,
                                 Y,
                                 train_size=0.67,
                                 test_size=0.33,
                                 random_state=0)

print ("X_raw_treino: ", X_raw_treino.shape)
print ("Y_treino: ", Y_treino.shape)
print ("X_raw_teste: ", X_raw_teste.shape)
print ("Y_teste: ", Y_teste.shape)
```

```
X_treino:  (240, 1)
Y_treino:  (240,)
X_teste:  (119, 1)
Y_teste:  (119,)
```

Avaliação Holdout na sklearn (6/14)

- **<u>PASSO 2 (parte 2)</u>: divisão da base de dados em treino e teste**
  - Verificando se houve estratificação

```
print('demonstrando a estratificação:')
print('\nfreqs. labels de treino:');
print(Y_treino.value_counts() * 100 / len(Y_treino))
print('\nfreqs. labels de teste:');
print(Y_teste.value_counts() * 100 / len(Y_teste))
```

```
freqs. labels de treino:
sentiment
negative    50.833333
positive    49.166667
Name: count, dtype: float64

freqs. labels de teste:
sentiment
positive    53.781513
negative    46.218487
Name: count, dtype: float64
```

Avaliação Holdout na sklearn (7/14)

- **Resultado ao final do Passo 2:** X_raw dividido em X_raw_treino e X_raw_teste; Y dividido em Y_treino e Y_teste

```
                                                  review
239                   Learning about lambda calculus :)
179  Obama's got JOKES!! haha just got to watch a b...
349  Monday already. Iran may implode. Kitchen is a...
322  SUPER INVESTORS: A great weekend read here fro...
200                       itchy back!! dont ya hate it!
..                                                   ...
323  reading Michael Palin book, The Python Years.....
192  I have to go to Booz Allen Hamilton for a 2hr ...
117  time warner has the worse customer service eve...
47   Highly recommend: http://tinyurl.com/HowDavidB...
172          The real AIG scandal / http://bit.ly/b82Px

[240 rows x 1 columns]
```

X_raw_treino

```
                                                  review
6    Fuck this economy. I hate aig and their non lo...
259    Only one exam left, and i am so happy for it :D
106  no watching The Night At The Museum. Getting R...
215     am loving new malcolm gladwell book - outliers
90   Goodby Silverstein agency new site! http://www...
..                                                   ...
81   It's a bank holiday, yet I'm only out of work ...
1    Reading my kindle2...  Love it... Lee childs i...
134  luke and i got stopped walking out of safeway ...
8                                         Loves twitter
260            Math review. Im going to fail the exam.

[119 rows x 1 columns]
```

X_raw_teste

```
239    positive
179    positive
349    negative
322    positive
200    negative
         ...
323    positive
192    negative
117    negative
47     positive
172    negative
```

Y_treino

```
6      negative
259    positive
106    positive
215    positive
90     positive
         ...
81     negative
1      positive
134    negative
8      positive
260    negative
```

Y_teste

Avaliação Holdout na sklearn (8/14)

- **<u>PASSO 3</u>: Pré-processamento das partes X de treino e X de teste**
  - Conversão cada texto dos DataFrames "X_raw_treino" e "X_raw_teste" p/ Bag of Words (BoW).
  - Gera as bag of words usando a classe `CountVectorizer` do módulo `sklearn.feature_extraction.text`. *É da mesma forma que fizemos em uma aula anterior no exemplo prático do Naive Bayes Multinomial.*
    - Primeiro devemos instanciar um objeto CountVectorizer e fazer nos dados de treino, usando o método fit_transform.
    - Depois devemos aplicar o mesmo objeto CountVectorizer nos dados de teste com o uso do método transform.
      - *Isso é necessário porque podem aparecer palavras na base de teste que não estavam no treino. Elas precisam ser descartadas, pois o modelo treinado desconhece essas palavras. Com o transform esse descarte é automático.*

Avaliação Holdout na sklearn (9/14)

- **<u>PASSO 3</u>: Pré-processamento das partes X de treino e X de teste**

```
from sklearn.feature_extraction.text import CountVectorizer

#-----------------------------------------------------------------
# (1) transformação da partição de treino em BoW com a classe
# CountVectorizer e o método fit_transform
#-----------------------------------------------------------------
c_vect = CountVectorizer()
bow_treino =
c_vect.fit_transform(X_raw_treino['review']).toarray()
tokens = c_vect.get_feature_names_out() # vocabulário de treino
X_treino = pd.DataFrame(bow_treino, columns=tokens)

#-----------------------------------------------------------------
# (2) transformação da partição de teste em BoW, aplicando o
# mesmo CountVectorizer e o método transform
#-----------------------------------------------------------------
bow_teste = c_vect.transform(X_raw_teste['review']).toarray()
X_teste = pd.DataFrame(bow_teste, columns=tokens)
```

Avaliação Holdout na sklearn (10/14)

- **<u>Resultado</u> ao final do <u>Passo 3:</u>** textos de treino e teste estruturados em suas respectivas BoWs

	10	1000	13t7nr	16	1988	19epah	19j2d	1st	1zlff	200	...	yes	yo	york	you	your	yr	yummmmmy	zealots	zero	zomg
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
235	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
236	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
237	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
238	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
239	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

240 rows × 1312 columns

X_treino

	10	1000	13t7nr	16	1988	19epah	19j2d	1st	1zlff	200	...	yes	yo	york	you	your	yr	yummmmmy	zealots	zero	zomg
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
114	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
115	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
116	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
117	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
118	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

119 rows × 1312 columns

X_teste

Avaliação Holdout na sklearn (11/14)

- **PASSO 4: Treinamento do classificador com os dados de treino**
  - (1). **importar o pacote** com o algoritmo que você quer usar, nesse caso o Perceptron (pacote `sklearn.naive_bayes` da scikit)
  - (2). **Instanciar o classificador**
    - Nesse caso, MultinomialNB.
  - (3). **Usar o método** (função) **fit**, para fazer o treinamento do classificador com os dados de treino:
    - X_treino
    - Y_treino

```
#1. importa o pacote com a classe MultinomialNB
from sklearn.naive_bayes import MultinomialNB

#2. instancia um objeto da classe MultinomialNB
modelo = MultinomialNB()

#3. utiliza o método "fit" para treinar o modelo
modelo.fit(X_treino, Y_treino)
```

Avaliação Holdout na sklearn (12/14)

- **<u>PASSO 5 (parte 1)</u>: Teste do modelo com a partição de teste**
  - Vamos começar aplicando o modelo aprendido na partição de teste, obtendo assim as predições do modelo

```
Y_predito = modelo.predict(X_teste)
```

```
['negative' 'positive' 'positive' 'positive' 'positive' 'negative'
 'negative' 'positive' 'negative' 'negative' 'positive' 'negative'
 'positive' 'positive' 'positive' 'positive' 'negative' 'negative'
 'positive' 'negative' 'positive' 'negative' 'positive' 'negative'
 'positive' 'positive' 'positive' 'negative' 'negative' 'negative'
 'positive' 'negative' 'negative' 'positive' 'negative' 'positive'
 'positive' 'negative' 'positive' 'negative' 'positive' 'negative'
 'negative' 'positive' 'negative' 'negative' 'negative' 'negative'
 'negative' 'positive' 'negative' 'negative' 'positive' 'positive'
 'positive' 'positive' 'positive' 'positive' 'positive' 'positive'
 'positive' 'positive' 'positive' 'negative' 'positive' 'negative'
 'negative' 'positive' 'negative' 'positive' 'negative' 'negative'
 'negative' 'negative' 'negative' 'negative' 'positive' 'negative'
 'positive' 'negative' 'negative' 'positive' 'positive' 'negative'
 'negative' 'negative' 'positive' 'positive' 'positive' 'positive'
 'negative' 'negative' 'positive' 'positive' 'positive' 'negative'
 'positive' 'negative' 'positive' 'positive' 'positive' 'positive'
 'positive' 'positive' 'negative' 'positive' 'negative' 'negative'
 'positive' 'positive' 'positive' 'negative' 'positive' 'positive'
 'negative' 'positive' 'negative' 'negative' 'negative']
```

Avaliação Holdout na sklearn (13/14)

- **<u>PASSO 5 (parte 2)</u>: Teste do modelo com a partição de teste**
  - De posse de Y_predito e Y_teste, podemos gerar a matriz de confusão
    - Usa-se a função confusion_matrix do módulo metrics.
    - Em nosso exemplo, a diagonal principal tem os maiores valores, o que significa que o classificador acertou a maioria das classificações

```
from sklearn import metrics

# matriz de confusão
mc = metrics.confusion_matrix(y_true = Y_teste,
                              y_pred = Y_predito)

print('Matriz de Confusão -', modelo.classes_, ':')
print(mc)
```

```
Matriz de Confusão - ['negative' 'positive'] :
[[46  9]
 [10 54]]
```

Avaliação Holdout na sklearn (14/14)

- **PASSO 5 (parte 3): Teste do modelo com a partição de teste**
  - Para obter os valores de acurácia, precisão, revocação e f1, basta usar as funções que calculam essas medidas (veja no notebook)
    - Temos que passar como parâmetros:
      - Y_teste (as classes reais),
      - Y_predito (as classes que o modelo inferiu)
      - Qual é a classe positiva do problema (no parâmetro pos_label)
      - E especificar que estamos trabalhando com classificação binária (no parâmetro binary)

```
# precisão = VP / (VP + FP)

f1 = metrics.f1_score(Y_teste, Y_predito,
                      pos_label = 'positive',
                      average = 'binary')
```

```
F1-Score: 0.850
```

IBGE
Escola Nacional de Ciências Estatísticas ENCE

EXERCÍCIOS

- (1) Modifique o notebook desta aula da seguinte maneira:
  - Treine um segundo modelo (modelo2) de classificação utilizando **Regressão Logística** (módulo sklearn.linear_model, classe LogisticRegression) usando a partição de treino.
  - Teste o modelo com a partição e teste e imprima a matriz de confusão e os valores das medidas de acurácia, precisão, recall e F1.
  - Qual modelo teve melhor desempenho? Naïve Bayes ou Regressão Logística

- (2) Na Matriz de Confusão abaixo, considere que **C1** é a **classe negativa** e **C2** a **classe positiva**. Calcule o valor das seguintes medidas de desempenho:
  - (a) Acurácia
  - (b) Precisão
  - (c) Revocação (*recall*)

		Rótulos Preditos	
	Classes	C1	C2
Rótulos Reais	C1	310	14
	C2	6	170

Referências

- sklearn.model_selection.train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

- sklearn.metrics.confusion_matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html#sklearn.metrics.confusion_matrix.

- scikit-learn - Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

- Split Your Dataset with scikit-learn's train_test_split. https://realpython.com/train-test-split-python-data/